```
> # ================================================================
======
# OptoFoton.mpl — TauTimp photon field in cylindrical
coordinates
# Author: Viorel (Teoria Onduscular / TauTimp)
# Companion to OptoFoton.md
# Plot variables: rho = r/R, zeta = z/R, tau = c*t/R
(dimensionless)
# ================================================================
======
restart:
with(plots):
with(plottools):
interface(rtablesize=40):

# ----------------------------------------------------------------
------
# SECTION 0 — Physical constants and TauTimp parameters
# ----------------------------------------------------------------
------
R_cosmo   := 3.9228e22:        # cosmological radius [m]
c_light   := 2.998e8:          # speed of light [m/s]
hbar      := 1.0546e-34:       # reduced Planck constant [J s]
eps0      := 8.854e-12:        # vacuum permittivity [F/m]
e_charge  := 1.602e-19:        # elementary charge [C]
m_e       := 9.109e-31:        # electron mass [kg] (used as
scale)

printf("Cosmological radius R       = %.4e m\n", evalf(R_cosmo)):
printf("Speed of light c            = %.4e m/s\n", evalf(c_light)
):
printf("Reduced Planck constant     = %.4e J s\n", evalf(hbar)):

# ----------------------------------------------------------------
------
# SECTION 1 — Symbolic statement of the master PDE
# ----------------------------------------------------------------
------
# Cylindrical Laplacian with axial symmetry (d/dtheta = 0):
#   Δ H = H_rr + (1/r) H_r + H_zz
#
# Master equation (TauTimp photon field):
```

```
#   (2m/hbar^2)[E + e^2/(4 pi eps0 r) - l(l+1) hbar^2/(2 m r^2)]
H
#   + (R/r) Δ H = (1/c^2) H_tt

printf("\n--- Master PDE assembled symbolically ---\n"):
printf("(2m/hbar^2)[E + e^2/(4 pi eps0 r) - l(l+1) hbar^2/(2 m
r^2)] H\n"):
printf("  + (R/r) Laplacian(H) = (1/c^2) d^2 H / d t^2\n"):

# ------------------------------------------------------------
------
# SECTION 2 — Separation of variables H = H3(r) H4(z) u(t)
# ------------------------------------------------------------
------
# u(t)  = exp(-i omega t)            -> (1/c^2) u''/u = -
omega^2/c^2
# H4(z) = exp(i kz z)                -> H4''/H4 = -kz^2
#
# Reduced radial ODE (Eq. 9 in the .md):
#   H3'' + (1/r) H3' + [alpha r + beta - gamma/r] H3 = 0
# with
#   alpha = (1/R)( 2 m E/hbar^2 + omega^2/c^2 )
#   beta  = 2 m e^2/(4 pi eps0 hbar^2 R) - kz^2
#   gamma = l(l+1)/R

printf("\n--- Radial ODE in canonical form (Eq. 9) ---\n"):
printf("  H3'' + (1/r) H3' + [alpha r + beta - gamma/r] H3 =
0\n"):
printf("alpha = (2 m E/hbar^2 + omega^2/c^2)/R\n"):
printf("beta  = 2 m e^2/(4 pi eps0 hbar^2 R) - kz^2\n"):
printf("gamma = l(l+1)/R\n"):
printf("Substitution H3 = r^(-1/2) chi(r) absorbs the first-
derivative term\n"):
printf("  -> chi'' + [alpha r + beta - gamma/r + 1/(4 r^2)] chi =
0  (Eq.10)\n"):

# ------------------------------------------------------------
------
# SECTION 3 — Bessel and Airy asymptotic regimes; eigenvalue
search
# ------------------------------------------------------------
------
```

```
# Bessel J0 zeros (boundary condition H3(R) = 0 in beta-dominated
regime)
j0_zeros_eval := [ evalf(BesselJZeros(0,1)),
                   evalf(BesselJZeros(0,2)),
                   evalf(BesselJZeros(0,3)),
                   evalf(BesselJZeros(0,4)),
                   evalf(BesselJZeros(0,5)) ]:
printf("\nFirst five zeros of J0:\n"):
for k from 1 to 5 do
    printf("  j(0,%d) = %.6f\n", k, evalf(j0_zeros_eval[k]))
end do:

# Airy Ai zeros (boundary condition for the Airy branch)
ai_zeros_num := [-2.3381, -4.0879, -5.5206, -6.7867, -7.9441]:
printf("\nFirst five zeros of Ai (negative real axis):\n"):
for k from 1 to 5 do
    printf("  a_%d = %.4f\n", k, evalf(ai_zeros_num[k]))
end do:

# Quantized radial wavenumber k_n = j(0,n)/R
k_n := proc(n) evalf(j0_zeros_eval[n] / R_cosmo) end proc:
printf("\nQuantized radial wavenumbers k_n = j(0,n)/R:\n"):
for n from 1 to 5 do
    printf("  k_%d = %.4e m^-1\n", n, evalf(k_n(n)))
end do:

# Photon angular frequencies omega_n = c * k_n (vacuum
dispersion)
omega_n_val := proc(n) evalf(c_light * k_n(n)) end proc:
printf("\nPhoton frequencies omega_n = c k_n:\n"):
for n from 1 to 5 do
    printf("  omega_%d = %.4e rad/s\n", n, evalf(omega_n_val(n)))
end do:

# ----------------------------------------------------------------
------
# SECTION 4 — Plot 1: Radial eigenfunctions H3(rho) = J0(j0_n *
rho)
# ----------------------------------------------------------------
------
# Use dimensionless rho = r/R as the plot variable.
```

```
H3_n := (n, rho) -> BesselJ(0, j0_zeros_eval[n] * rho):

plot_H3 := plot(
    [ H3_n(1, rho), H3_n(2, rho),
      H3_n(3, rho), H3_n(4, rho) ],
    rho = 0..1,
    color  = [blue, red, green, "Orange"],
    legend = ["n=1", "n=2", "n=3", "n=4"],
    thickness = 2,
    title  = "Radial eigenfunctions H_3(r) = J_0(k_n r)",
    labels = ["r/R", "H_3"],
    gridlines = true
):

# ------------------------------------------------------------------
------
# SECTION 5 — Plot 2: Axial mode H4(zeta) = cos(Kz * zeta),  Kz
= kz*R
# ------------------------------------------------------------------
------
# zeta = z/R is dimensionless; Kz absorbs the R factor.

H4_kz := (Kz, zeta) -> cos(Kz * zeta):

plot_H4 := plot(
    [ H4_kz(1, zeta), H4_kz(3, zeta),
      H4_kz(5, zeta), H4_kz(8, zeta) ],
    zeta = -1..1,
    color  = [blue, red, green, "Orange"],
    legend = ["k_z R = 1", "k_z R = 3", "k_z R = 5", "k_z R =
8"],
    thickness = 2,
    title  = "Axial mode H_4(z) = cos(k_z z)",
    labels = ["z/R", "H_4"],
    gridlines = true
):

# ------------------------------------------------------------------
------
# SECTION 6 — Plot 3: |psi|^2 = J0^2(k_1 r) cos^2(Kz z)  at t=0
# ------------------------------------------------------------------
------
```

```
psi_sq := (rho, zeta) -> BesselJ(0, j0_zeros_eval[1]*rho)^2 * cos
(3*zeta)^2:

plot_psi2 := plot3d(
    psi_sq(rho, zeta),
    rho = 0..1, zeta = -1..1,
    axes   = boxed,
    style  = surfacecontour,
    colorscheme = ["zgradient", ["DarkBlue", "Cyan", "Yellow",
"Red"]],
    title  = "Probability density |psi|^2 at t=0",
    labels = ["r/R", "z/R", "|psi|^2"],
    grid   = [60, 60]
):

# ------------------------------------------------------------
------
# SECTION 7 — Plot 4: Independent Gaussian P(r,z,0)
# ------------------------------------------------------------
------
# P(r,z,0) = A exp[-(rho-rho0)^2/(2 sig_r^2)] exp[-zeta^2/(2
sig_z^2)]
A_gauss := 1.0:
rho0    := 0.5:           # r0 = R/2 in normalized units
sig_r   := 0.1:           # sigma_r = R/10
sig_z   := 0.1:           # sigma_z = R/10

P_gauss := (rho, zeta) -> A_gauss
   * exp(-(rho - rho0)^2 / (2*sig_r^2))
   * exp(- zeta^2        / (2*sig_z^2)):

# Boundary check at (R, 0, 0): rho=1, zeta=0
P_at_boundary := evalf(P_gauss(1.0, 0.0)):
printf("\nBoundary check: P(R, 0, 0) = %.4e  (should be ~0)\n",
       evalf(P_at_boundary)):

plot_Pgauss := plot3d(
    P_gauss(rho, zeta),
    rho = 0..1, zeta = -1..1,
    axes   = boxed,
    style  = surfacecontour,
    colorscheme = ["zgradient", ["DarkBlue", "RoyalBlue", "Cyan",
```

```
                                    "Yellow", "Orange", "Red"]],
    title  = "Gaussian P(r,z,0): r0=R/2, sigma=R/10",
    labels = ["r/R", "z/R", "P(r,z,0)"],
    grid   = [60, 60]
):

# ------------------------------------------------------------
------
# SECTION 8 — Plot 5: d'Alembert time-evolution snapshots
#             P(r,z,t) = (A/2)[ G(r-ct) + G(r+ct) ] * G_z(z)
# ------------------------------------------------------------
------
# tau = c t / R (dimensionless time)

G_pulse := (rho, tau) -> 0.5 * (
    exp(-(rho - tau - rho0)^2 / (2*sig_r^2))
  + exp(-(rho + tau - rho0)^2 / (2*sig_r^2)) ):

P_dalembert := (rho, zeta, tau) -> A_gauss * G_pulse(rho, tau)
                                   * exp(-zeta^2 / (2*sig_z^2)):

# Five time snapshots: tau = 0, 0.1, 0.2, 0.3, 0.4 along z = 0
plot_dal := plot(
    [ P_dalembert(rho, 0, 0.0),
      P_dalembert(rho, 0, 0.1),
      P_dalembert(rho, 0, 0.2),
      P_dalembert(rho, 0, 0.3),
      P_dalembert(rho, 0, 0.4) ],
    rho = 0..1,
    color  = [black, blue, green, "Orange", red],
    legend = ["tau=0.0", "tau=0.1", "tau=0.2", "tau=0.3", "tau=
0.4"],
    thickness = 2,
    title  = "d'Alembert evolution P(r,0,tau), tau = c t / R",
    labels = ["r/R", "P"],
    gridlines = true
):

# ------------------------------------------------------------
------
# SECTION 9 — Combined 2x3 panel display
# ------------------------------------------------------------
```

```
------
panel := display(
    Array([ [plot_H3,     plot_H4,    plot_psi2 ],
            [plot_Pgauss, plot_dal,   plot_H3  ] ])
):

print(panel):

# ------------------------------------------------------------
------
# SECTION 10 — Eigenvalue summary table
# ------------------------------------------------------------
------
printf("\n============================================================
==\n"):
printf(" Eigenvalue summary (Bessel branch, beta-dominated
regime)\n"):
printf("============================================================
\n"):
printf("  n  |  j(0,n)   |   k_n [m^-1]   |  omega_n [rad/s]\n"):
printf("------------------------------------------------------------
\n"):
for n from 1 to 5 do
    printf("  %d  | %8.4f  |   %.4e   |   %.4e\n",
           n,
           evalf(j0_zeros_eval[n]),
           evalf(k_n(n)),
           evalf(omega_n_val(n)) )
end do:
printf("============================================================
\n"):

# ------------------------------------------------------------
------
# SECTION 11 — Maxwell dispersion check (Option A)
# ------------------------------------------------------------
------
printf("\nMaxwell dispersion verification: omega^2 = c^2 k^2\n"):
for n from 1 to 5 do
    lhs_disp := evalf( omega_n_val(n)^2 ):
    rhs_disp := evalf( c_light^2 * k_n(n)^2 ):
    ratio    := evalf( lhs_disp / rhs_disp ):
```

```
    printf("  n=%d: omega^2/(c^2 k^2) = %.6f\n", n, evalf(ratio))
  end do:

  # ------------------------------------------------------------------
  ------
  # END OF FILE
  # ------------------------------------------------------------------
  ------
  printf("\nOptoFoton.mpl execution complete.\n"):

Cosmological radius R       = 3.9228e+22 m
Speed of light c            = 2.9980e+08 m/s
Reduced Planck constant     = 1.0546e-34 J s

--- Master PDE assembled symbolically ---
(2m/hbar^2)[E + e^2/(4 pi eps0 r) - l(l+1) hbar^2/(2 m r^2)] H
  + (R/r) Laplacian(H) = (1/c^2) d^2 H / d t^2

--- Radial ODE in canonical form (Eq. 9) ---
  H3'' + (1/r) H3' + [alpha r + beta - gamma/r] H3 = 0
alpha = (2 m E/hbar^2 + omega^2/c^2)/R
beta  = 2 m e^2/(4 pi eps0 hbar^2 R) - kz^2
gamma = l(l+1)/R
Substitution H3 = r^(-1/2) chi(r) absorbs the first-derivative term
  -> chi'' + [alpha r + beta - gamma/r + 1/(4 r^2)] chi = 0   (Eq.10)

First five zeros of J0:
  j(0,1) = 2.404826
  j(0,2) = 5.520078
  j(0,3) = 8.653728
  j(0,4) = 11.791534
  j(0,5) = 14.930918

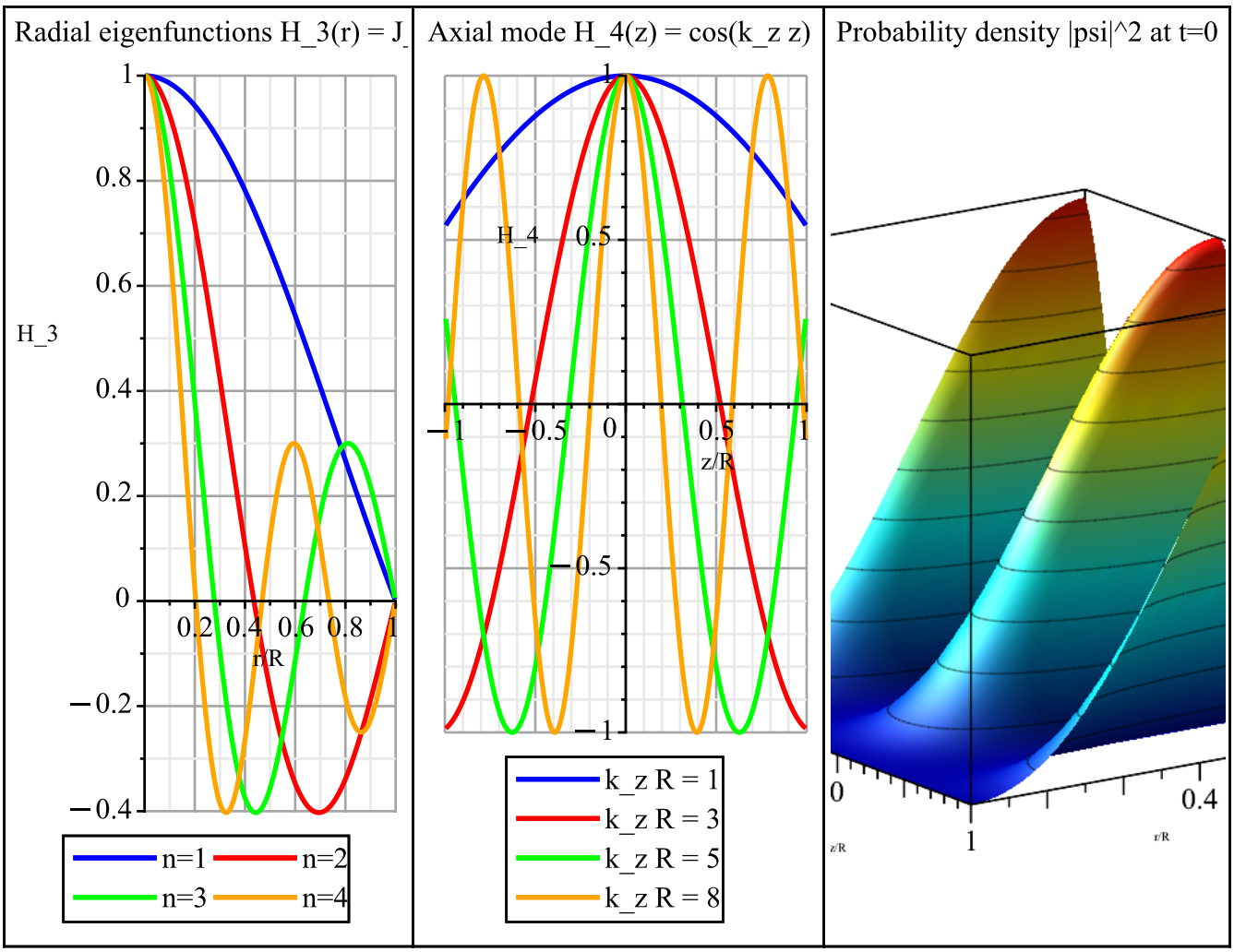
First five zeros of Ai (negative real axis):
  a_1 = -2.3381
  a_2 = -4.0879
  a_3 = -5.5206
  a_4 = -6.7867
  a_5 = -7.9441

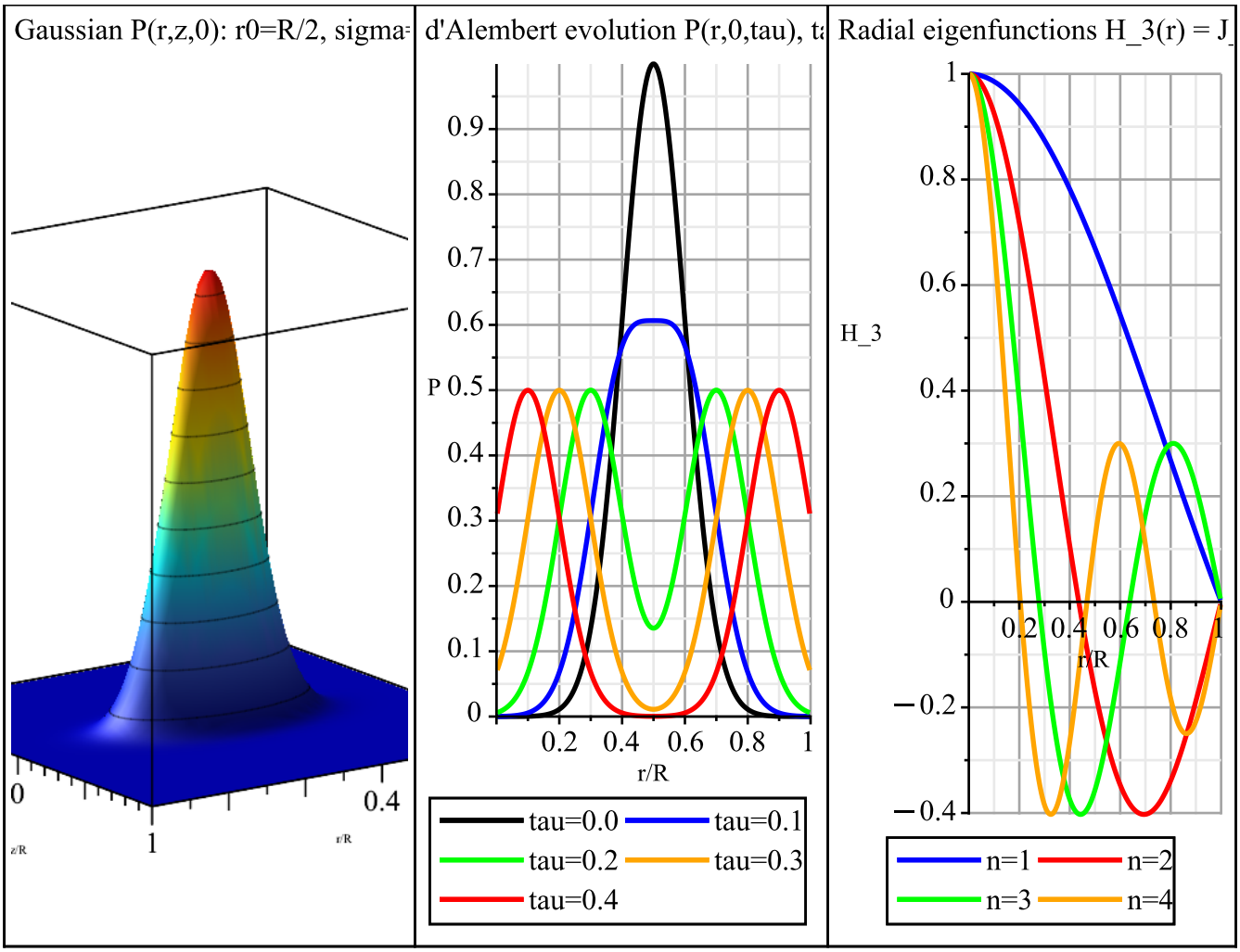
Quantized radial wavenumbers k_n = j(0,n)/R:
  k_1 = 6.1304e-23 m^-1
  k_2 = 1.4072e-22 m^-1
```

```
  k_3 = 2.2060e-22 m^-1
  k_4 = 3.0059e-22 m^-1
  k_5 = 3.8062e-22 m^-1

Photon frequencies omega_n = c k_n:
  omega_1 = 1.8379e-14 rad/s
  omega_2 = 4.2187e-14 rad/s
  omega_3 = 6.6136e-14 rad/s
  omega_4 = 9.0117e-14 rad/s
  omega_5 = 1.1411e-13 rad/s

Boundary check: P(R, 0, 0) = 3.7267e-06  (should be ~0)
```






```
============================================================
 Eigenvalue summary (Bessel branch, beta-dominated regime)
============================================================
  n   |  j(0,n)    |   k_n [m^-1]    |  omega_n [rad/s]
------------------------------------------------------------
  1   |   2.4048   |   6.1304e-23    |   1.8379e-14
  2   |   5.5201   |   1.4072e-22    |   4.2187e-14
  3   |   8.6537   |   2.2060e-22    |   6.6136e-14
  4   |  11.7915   |   3.0059e-22    |   9.0117e-14
  5   |  14.9309   |   3.8062e-22    |   1.1411e-13
============================================================

Maxwell dispersion verification: omega^2 = c^2 k^2
  n=1: omega^2/(c^2 k^2) = 1.000000
  n=2: omega^2/(c^2 k^2) = 1.000000
  n=3: omega^2/(c^2 k^2) = 1.000000
  n=4: omega^2/(c^2 k^2) = 1.000000
```

```
  n=5: omega^2/(c^2 k^2) = 1.000000

OptoFoton.mpl execution complete.
```